Universidad Politécnica Metropolitana del Estado de Hidalgo (UPMH)

Maestría en Inteligencia Artificial · Asignatura: Ciberseguridad · Periodo 2026

Auditoría y mejora de un sistema de cifrado AES con apoyo de inteligencia artificial

Alumna: Jessica Melani Romero Lora

Profesor: Dr. Antonio Cedillo Hernández

Link al repositorio: https://github.com/jessicaromero-ctrl/Cibersecurity/tree/main/corte-02/practicas/aes-cifrado-auditoria

Aviso: documento con fines exclusivamente académicos. Los datos son ficticios y las técnicas deben aplicarse únicamente en entornos aislados y autorizados.

1. Objetivo y alcance

Analizar una implementación de AES-128 en modo ECB, identificar sus vulnerabilidades con apoyo de herramientas de inteligencia artificial y sustituirla por una implementación con AES-GCM que garantice confidencialidad e integridad. Se verifica el comportamiento mediante una prueba de manipulación del texto cifrado.

2. Ejecución del sistema original (Parte 1)

Pregunta 1: ¿Qué información puede observarse en la variable cifrado?

La variable cifrado contiene el resultado de cipher.encrypt(pad(mensaje.encode(), 16)): una secuencia de bytes que no es legible como texto. Su longitud es múltiplo de 16 porque el relleno PKCS#7 completa el último bloque. Al imprimirla en Base64 se representan esos bytes con caracteres imprimibles, pero eso no es cifrado. Por sí misma, la variable no revela el mensaje, aunque sí deja ver su longitud en bloques y, en ECB, los patrones entre bloques repetidos.

Pregunta 2: ¿El programa logra recuperar correctamente el mensaje original?

Sí. El descifrado usa la misma clave y el modo ECB. decrypt devuelve los bloques con relleno y unpad elimina los bytes añadidos, por lo que el mensaje recuperado coincide con el original. Que el mensaje se recupere demuestra que el cifrado es reversible con la clave, pero no que el sistema proteja la información.

Pregunta 3: ¿El hecho de que el programa funcione significa necesariamente que sea seguro?

No. Que el programa funcione demuestra su funcionalidad, no su seguridad. La protección depende de tres factores: el algoritmo, el modo de operación y la gestión de la clave. En este código, la clave está escrita en el archivo, por lo que quien tenga acceso al programa puede descifrar toda la información. Además, ECB no verifica la integridad: alguien puede modificar el texto cifrado sin conocer la clave y el programa no lo detectará. Tampoco existe autenticación de quien opera el sistema, porque cualquier persona con la clave puede cifrar y descifrar sin restricción. AES es robusto, pero la seguridad depende de cómo se implementa.

Concepto	**Resultado**
Mensaje original	Información confidencial del cliente
Longitud en bytes (UTF-8)	37
Longitud tras relleno PKCS#7	48 bytes (3 bloques)
Mensaje recuperado	Igual al original

Tabla 1. Resultados de la ejecución del sistema original (AES-ECB).

3. Auditoría con inteligencia artificial (Parte 2)

El prompt de auditoría utilizado se incluye en el Anexo A. Las observaciones de la herramienta de IA fueron revisadas y contrastadas con el análisis propio.

Pregunta 4: ¿Cuáles son los tres riesgos o debilidades identificados?

Los tres riesgos identificados son los siguientes. Primero, la clave está hardcodeada en el código y tiene baja entropía: sigue un patrón predecible, por lo que cualquiera con acceso al archivo o a un respaldo puede descifrar la información. Segundo, el modo ECB cifra bloques de forma independiente, de modo que bloques de texto plano idénticos producen bloques cifrados idénticos. Esto revela patrones y la estructura del mensaje sin necesidad de descifrarlo. Tercero, ECB no verifica la integridad del texto cifrado, por lo que una alteración pasa desapercibida. El primero es un problema de gestión de claves; el segundo, de modo de operación; el tercero, de verificación de datos.

#	**Riesgo**	**Descripción**	**Impacto**
R1	Clave hardcodeada y de baja entropía	La clave está en el código y sigue un patrón predecible.	Quien acceda al código o a un respaldo puede descifrar la información.
R2	Modo ECB	Bloques de texto plano idénticos producen bloques cifrados idénticos.	Se revelan patrones y la estructura del mensaje.
R3	Ausencia de integridad	ECB no verifica si el texto cifrado fue modificado.	Una alteración pasa desapercibida.

Tabla 2. Riesgos identificados en la implementación original.

Pregunta 5: ¿Por qué utilizar una clave fija dentro del código representa un riesgo?

La clave fija representa un riesgo porque está almacenada junto al programa. Cualquier persona con acceso al código fuente, al repositorio o a un respaldo puede descifrar la información protegida. Además, al no cambiar nunca, una sola filtración expone tanto los datos pasados como los futuros. Por ello, la clave debe generarse de forma aleatoria, almacenarse fuera del código y rotarse periódicamente.

Pregunta 6: ¿Qué problema presenta el modo de operación ECB?

El modo ECB cifra cada bloque de forma independiente. Por ello, bloques de texto plano idénticos producen bloques cifrados idénticos. Esto permite identificar patrones y estructura del mensaje sin conocer la clave, lo que debilita la confidencialidad. Además, ECB no ofrece protección contra modificaciones del texto cifrado.



Figura 1. Comparación del efecto de ECB y GCM sobre bloques repetidos.

4. Mejora con AES-GCM (Parte 3)

Pregunta 7: ¿Cuál es la principal diferencia entre AES-ECB y AES-GCM?

La principal diferencia es que ECB no utiliza valores aleatorios: bloques de texto plano idénticos producen bloques cifrados idénticos, lo que revela patrones. GCM utiliza un nonce distinto en cada cifrado, por lo que el mismo mensaje produce cifrados diferentes. Además, GCM funciona en modo contador y no requiere relleno, de modo que el texto cifrado tiene la misma longitud que el mensaje, mientras que ECB debe

completar bloques. GCM también genera una etiqueta de autenticación, que ECB no produce.

Pregunta 8: ¿Qué función cumple la etiqueta de autenticación en GCM?

La etiqueta de autenticación permite verificar que el texto cifrado y el nonce no fueron modificados y que fueron generados con la clave. Si cambia un solo byte del cifrado, la función decrypt_and_verify rechaza el mensaje y no devuelve texto plano. La etiqueta protege la integridad y la autenticidad, pero no la confidencialidad, que corresponde al cifrado. No indica en qué byte ocurrió la modificación ni cuándo, porque se calcula sobre el mensaje completo.

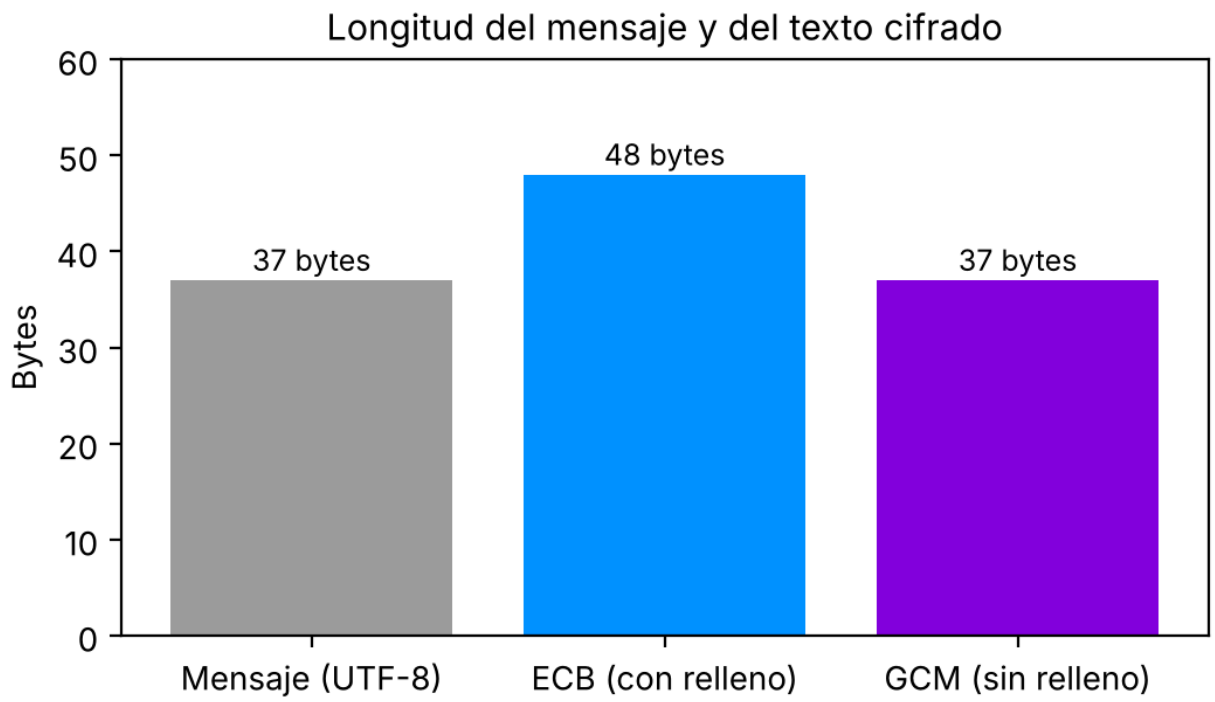


Figura 2. Longitud del mensaje y del texto cifrado en ECB y GCM.

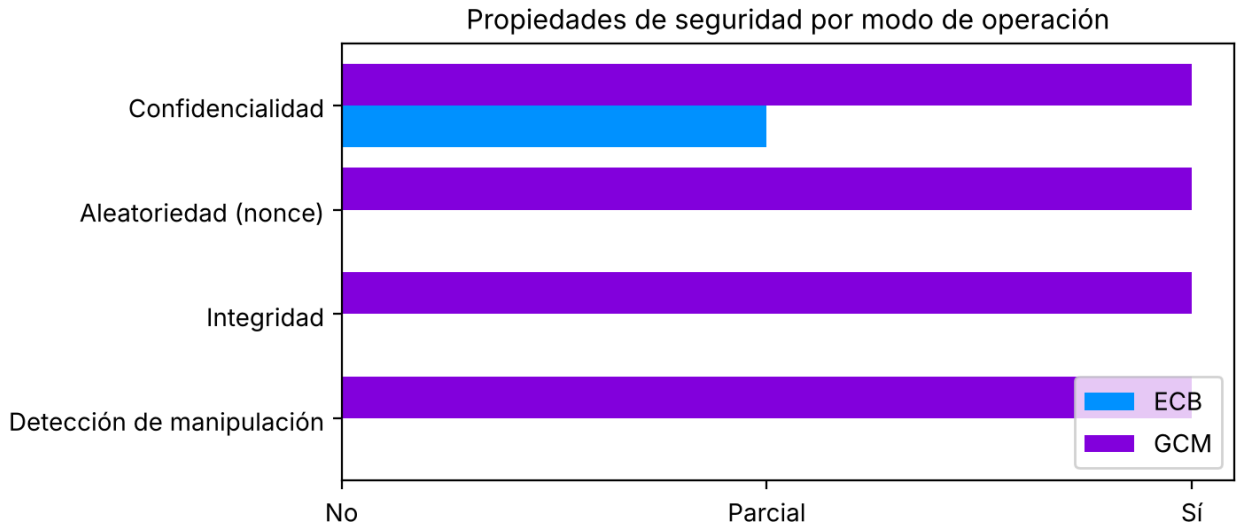


Figura 3. Propiedades de seguridad de ECB y GCM (0 = no, 1 = parcial, 2 = sí).

5. Prueba de manipulación (Parte 4)

Se modificó el primer byte del texto cifrado conservando la etiqueta original. El prompt correspondiente se incluye en el Anexo C.

Caso	Entrada	Resultado
Control	Texto cifrado original	Mensaje recuperado correctamente
Prueba	Texto cifrado con un byte alterado	ValueError: falló la verificación de autenticidad

Tabla 3. Resultados de la prueba de manipulación.

Pregunta 9: ¿Qué ocurrió cuando intentaste descifrar el mensaje modificado?

Al descifrar el texto alterado, decrypt_and_verify lanzó una excepción ValueError y el programa mostró el mensaje de error en lugar del texto plano. El byte modificado cambia el cálculo de la etiqueta de autenticación: la etiqueta original ya no coincide con el texto cifrado recibido, por lo que la función rechaza el mensaje antes de devolver cualquier resultado. Esto demuestra que la alteración de un solo byte es suficiente para ser detectada.

Pregunta 10: ¿Por qué es importante verificar la integridad y autenticidad de la información, además de proteger su confidencialidad?

La confidencialidad impide que alguien lea la información, pero no garantiza que nadie la altere. Sin integridad, un atacante podría modificar el texto cifrado y producir datos corruptos o manipulados sin conocer la clave. La autenticidad confirma que el mensaje

fue generado por quien posee la clave. Sin embargo, en un esquema simétrico como AES-GCM, cualquier poseedor de la clave puede crear mensajes válidos, por lo que la autenticidad prueba pertenencia a quien tiene la clave, no la identidad individual. Por ello, la protección de la clave es tan importante como la elección del algoritmo.

6. Conclusión

Un algoritmo reconocido como AES no es suficiente para garantizar la seguridad de un sistema. La práctica demostró que la robustez del algoritmo depende de decisiones de implementación: una clave fija en el código, un modo ECB que revela patrones y la ausencia de verificación de integridad anulan cualquier fortaleza matemática. Para proteger una plataforma se requieren controles complementarios, como autenticación multifactor, gestión segura y rotación de claves, y verificación de integridad mediante AES-GCM. La IA generativa acelera el desarrollo de código criptográfico, pero también introduce riesgos: produce soluciones plausibles que pueden ser inseguras, y puede ser aprovechada para analizar implementaciones deficientes. Por ello, el código criptográfico debe revisarse con fundamentos matemáticos y auditorías humanas.

Anexo A. Prompt de auditoría (Parte 2)

Actúa como un especialista en ciberseguridad y criptografía. Analiza este código Python que utiliza AES para proteger información confidencial. Identifica tres riesgos o debilidades de seguridad y explica cada uno de forma sencilla. Propón alternativas más seguras y justifica tus recomendaciones. No modifiques el código todavía.

Anexo B. Prompt de modificación con AES-GCM (Parte 3)

Ahora modifica el programa original para utilizar AES-128 en modo GCM en lugar de ECB. El programa debe generar una clave aleatoria de 128 bits, utilizar un nonce adecuado, cifrar el mensaje y recuperarlo mediante una operación de descifrado que verifique la etiqueta de autenticación. Explícame brevemente qué cambiaste y por qué. El código debe poder ejecutarse en Google Colab utilizando PyCryptodome.

Anexo C. Prompt de la prueba de manipulación (Parte 4)

Agrega una prueba a mi programa AES-GCM que modifique un byte del texto cifrado antes de intentar descifrarlo. El programa debe conservar la etiqueta de autenticación original y utilizar decrypt_and_verify() para detectar la modificación. Captura la excepción correspondiente y muestra un mensaje indicando que falló la verificación de autenticidad.